

UNIT-5

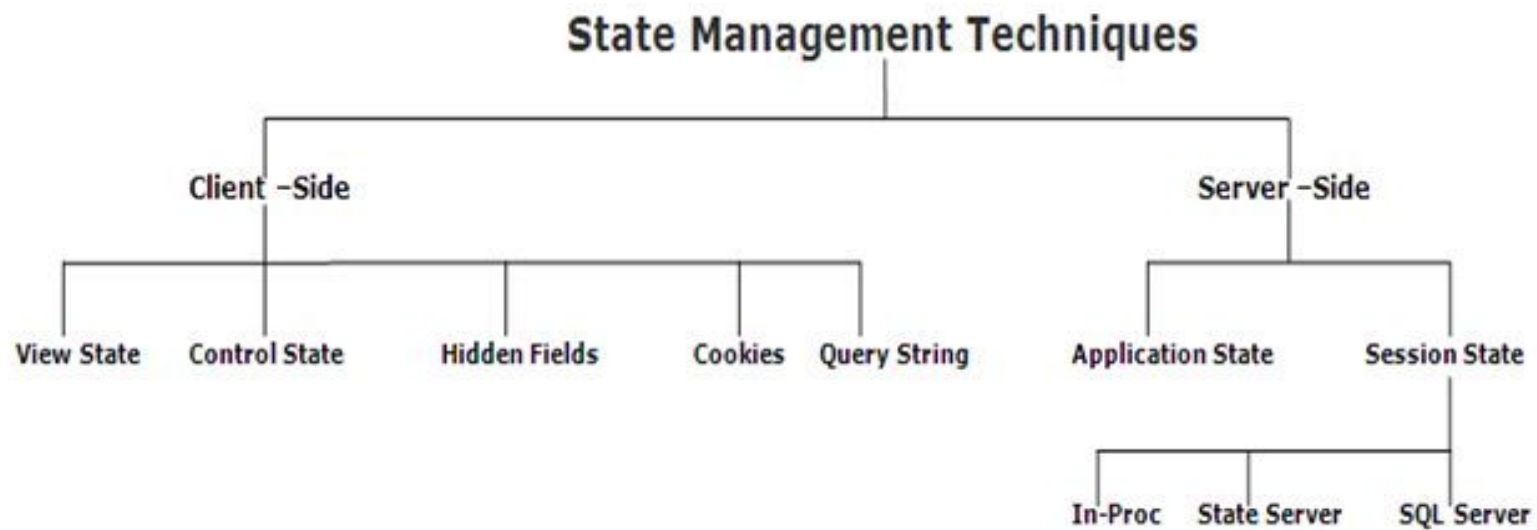
State Management

- State management means page, object/data, and user in the application explicitly because all ASP.NET web applications are stateless, i.e., by default, for each page posted to the server, the state of controls is lost. Nowadays all web apps demand a high level of state management from control to application level.
- HTTP/HTTPS doesn't remember what website or URL we visited, or in other words we can say it doesn't hold the state of a previous website that we visited before closing our browser, that is called stateless.

State Management

- ASP.NET Core MVC provides a rich set of features for building modern web applications, and these include support for a number of ways to manage state.
- State management is the technique of maintaining the state of an application over time, i.e., for the duration of a user session or across all of the HTTP requests and responses that constitute the session.
- Thus it is one of the most important cross-cutting concerns of any web application.

State Management



State management

- . Types of state management
 1. There are two types of state management techniques: client side and server side.

Client side

Cookies

HiddenField

Query Strings

Server side

Application

Session

TempData

Client-Side State Management

- Whenever we use Client-Side State Management, the state related information will directly get stored on the client-side. That specific information will travel back and communicate with every request generated by the user then afterwards provides responses after server-side communication.
- This architecture is something like the following,



Server-Side State Management

- Server-Side State Management is different from Client-Side State Management but the operations and working is somewhat the same in functionality. In Server-Side State Management all the information is stored in the user memory. Due to this functionality there is more secure domains at the server side in comparison to Client-Side State Management.
- The structure is something like the following,



Cookies

- A set of Cookies is a small text file that is stored in the user's hard drive using the client's browser. Cookies are just used for the sake of the user's identity matching as it only stores information such as sessions id's, some frequent navigation or post-back request objects.
- Whenever we get connected to the internet for accessing a specific service, the cookie file is accessed from our hard drive via our browser for identifying the user. The cookie access depends upon the life cycle or expiration of that specific cookie file.

- **Cookie | Types**

1. **Persistent Cookie**
2. **Non-Persistent Cookie**

Persistent Cookie

- Cookies having an expiration date is called a persistent cookie. This type of cookie reaches their end as their expiration dates comes to an end. In this cookie we set an expiration date.

```
CookieOptions options = new CookieOptions();
```

```
options.Expires = DateTime.Now.AddSeconds(10);
```

Non-Persistent Cookie

- Non-Persistent cookies are temporary. They are also called in-memory cookies and session-based cookies.
- These cookies are active as long as the browser remains active, in other words if the browser is closed then the cookies automatically expire
- It maintains user information as long as the user access or uses the services. Its simply the opposite procedure of a persistent cookie.

cookies

- Some features of cookies are:
 1. Store information temporarily
 2. It's just a simple small sized text file
 3. Can be changed depending on requirements
 4. User Preferred
 5. Requires only a few bytes or KBs of space for creating cookies

Hidden Field

- When working on ASP.NET Core MVC applications, we may need to preserve data on the client side instead of presenting it on the page.
- For example, we might need to send data to the server when the user takes a certain action, without showing the data in the user interface.
- This is a typical problem in many applications, and hidden fields offer an excellent solution. We can store information in hidden form fields and return it in the following request.
- `@Html.HiddenFor(x => x.UserId, new { Value = "1" })`

Query Strings

- **You can take advantage of query strings to transmit a small amount of data from one request to another.**
- **Note that because query strings are publicly exposed, you should never use them to pass sensitive data.**
- **Additionally, using query strings could make your application vulnerable to cross-site request forgery (CSRF) attacks.**
- **Points to Remember**
 - It is generally used for holding values
 - Works temporarily
 - Switches info from one to another page
 - Increase performance
 - Uses real and virtual path values for URL routing

Query Strings

A query string is data appended to a URL after a ? and passed as key–value pairs.

Syntax:

URL?key1=value1&key2=value2

Example:

<http://localhost:5655/api/customer?region=abc>

And, the code snippet below shows how you can read the query string data in your action method.

```
string region = HttpContext.Request.Query["region"].ToString();
```

Session

- . Session state is a mechanism for storing user data on the server side in an ASP.NET Core MVC web application.
- . A user's browser sends the server a request containing information about the user's session every time the user visits a website.
- . The server then creates a new session and stores the user's data in that session.
- . `HttpContext.Session.SetString("name", name);`
- . `RedirectToAction("list", "Student");`

Session

- The session is stored in the following for ways in ASP.NET.
- **InProcMode**
- It is a default session mode and a value store in web server memory (IIS). In this the session value stored with server start and it ends when the server is restarted.
- **State Server Mode**
- In this mode session data is stored in separate server.

Session

- **SQL Server Mode**
- In this session is stored in the database. It is a secure mode.
- **Custom Mode**
- Generally under session data is stored in InProc, Sql Server, State server, etc. If you store session data with other new techniques then provide ASP.NET.

Application

- Application State is a server side management state. It is also called application level state management. In this mainly store user activity in server memory and application event shown in Global.asax file.
- There are three stages of applications in ASP.NET.
- Application_Start
- This event begins with domain start.
- `Void Application_Start(object sender, EventArgs e)`
- `{`
- `Application["AppstartMessage"] = "Welcome to Developer Community";`
- `}`

Application

- Application_Error
- void Application_Error(object sender, EventArgs e)
- {
- // Write an unhandled error code exception
- }

Application

- Application_End
- This ends with domain or restarts IIS.
- `Void Application_End(object sender, EventArgs e)`
- `{`
- `Application["AppEndMessage"] = "Application Closed";`
- `}`

ViewBag, ViewData And TempData In MVC

- ViewBag, ViewData, and TempData are all objects in ASP.NET MVC, and these are used to pass the data in various scenarios.
 - The following are the scenarios where we can use these objects.
- ☐ Pass the data from Controller to View.
 - ☐ Pass the data from one action to another action in the same Controller.
 - ☐ Pass the data in between Controllers.
 - ☐ Pass the data between consecutive requests.

ViewBag, ViewData And TempData In MVC

- What is ViewBag?
- ViewBag is a dynamic object to pass the data from the Controller to View. This will pass the data as a property of the object ViewBag.
- And we have no need to typecast to read the data or for null checking. The scope of ViewBag is permitted to the current request, and the value of ViewBag will become null while redirecting.
- ```
Public ActionResult Index() {
```
- ```
    ViewBag.Title = "Welcome";
```
- ```
 return View();
```
- ```
}
```
- ```
<h2>@ViewBag.Title</h2>
```

# ViewBag, ViewData And TempData In MVC

- What is ViewData?
- ViewData is a dictionary object to pass the data from Controller to View, where data is passed in the form of a key-value pair.
- Typecasting is required to read the data in View if the data is complex, and we need to ensure a null check to avoid null exceptions.
- The scope of ViewData is similar to ViewBag, and it is restricted to the current request, and the value of ViewData will become null while redirecting.
- ```
Public ActionResult Index() {
```
- ```
 ViewData["Title"] = "Welcome";
```
- ```
    return View();
```
- ```
}
```
- ```
<h2>@ViewData["Title"]</h2>
```

ViewBag, ViewData And TempData In MVC

- What is TempData?
- TempData is a dictionary object to pass the data from one action to another action in the same Controller or different Controllers.
- Usually, the TempData object will be stored in a session object. TempData is also required to be typecast and for null checking before reading data from it.
- TempData scope is limited to the next request, and if we want TempData to be available even further, we should use Keep and Peek.

ViewBag, ViewData And TempData In MVC

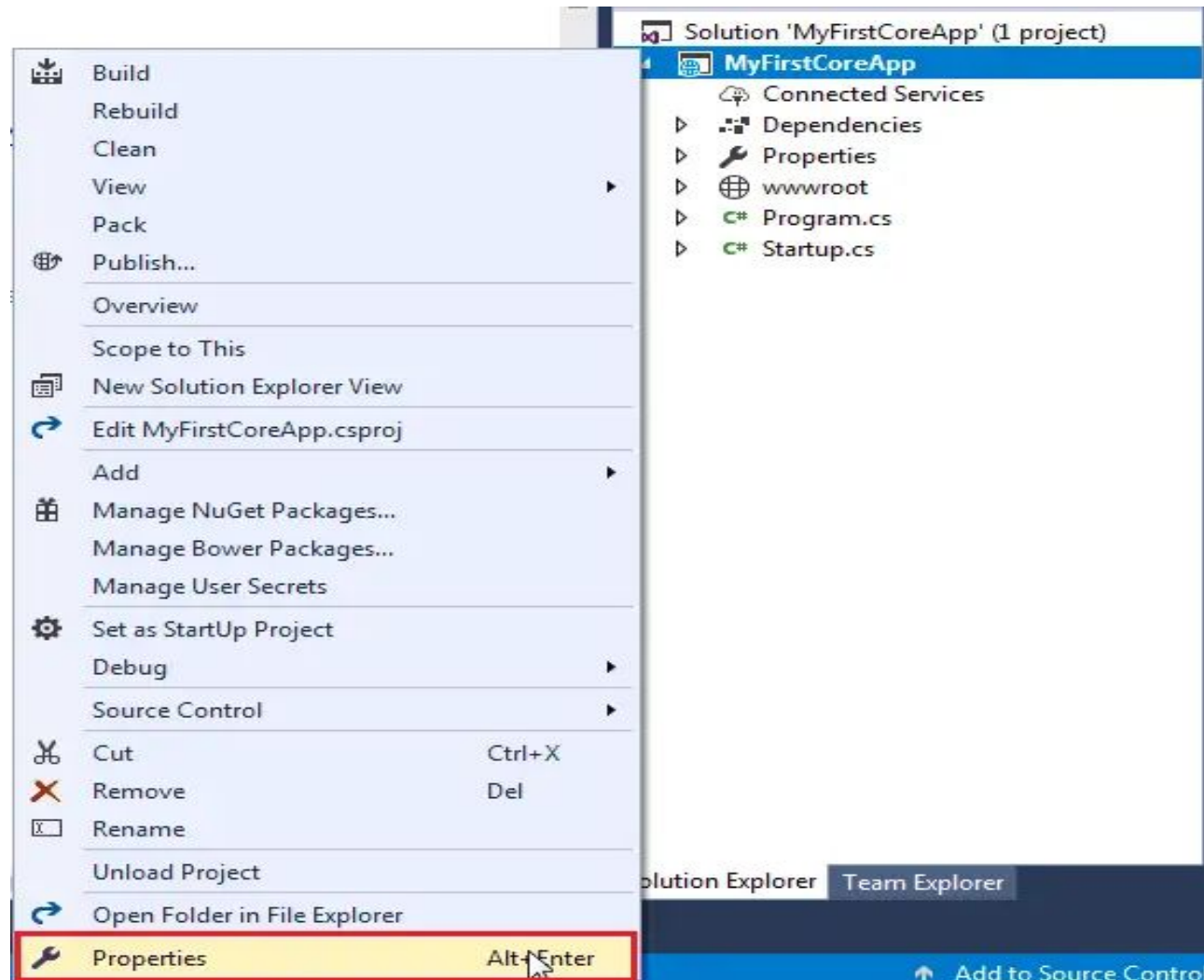
- `Public ActionResult Index()`
- `{`
- `TempData["Data"] = "I am from Index action";`
- `return View();`
- `}`

- `Public string Get()`
- `{`
- `return TempData["Data"] ;`
- `}`

ASP.NET Core - Environment Variable

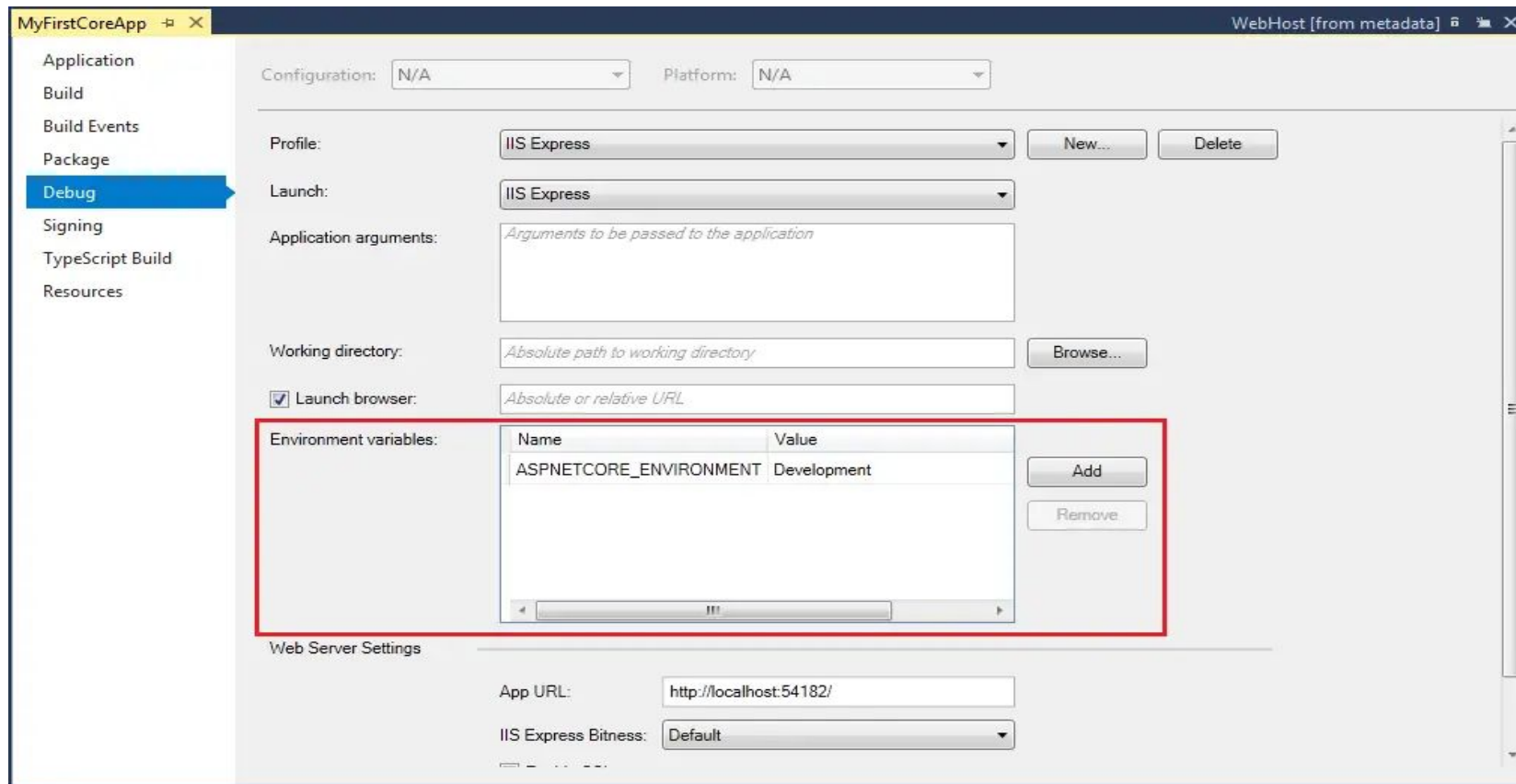
- ASP.NET Core uses an environment variable called `ASPNETCORE_ENVIRONMENT` to indicate the runtime environment.
- The value of this variable can be anything as per your need but typically it can be Development, Staging, or Production.
- The value is case insensitive in Windows and Mac OS but it is case sensitive on Linux.
- In Visual Studio, we can set `ASPNETCORE_ENVIRONMENT` in the debug tab of project properties. Open project properties by right clicking on the project in the solution explorer and select Properties.

ASP.NET Core - Environment Variable



ASP.NET Core - Environment Variable

- This will open properties page. Click on Debug tab and you will see Environment Variables as shown below.



Multiple Environments and Development Mode

- By letting the application know which environment it's running, you can vary the application's behavior. ASP.NET Core makes it easy to manage various environments effortlessly.
- You can configure different configuration settings for different environments, and tweak them without having to recompile the application. This lets you easily change the environment.
- There are three parts to managing environments for your application, identifying the environment, loading different configuration settings based on the environment, and changing the environment.
- To determine the runtime environment, ASP.NET Core uses environment variables named `DOTNET_ENVIRONMENT` or `ASPNETCORE_ENVIRONMENT` when the `ConfigureWebHostDefaults` method is called. The second environment variable overrides the first one.

Multiple Environments and Development Mode

- If the application doesn't find an `ASPNETCORE_ENVIRONMENT` variable when it starts, it defaults to a production environment.
- That means, unless you set up the variable on the deployment machine explicitly to development, all ASP.NET Core applications start in the production mode.
- You can also use the `launchSettings.json` file to set up the hosting environment. This file resides in the Properties folder, and defines profiles for running the application.
- Here's an example of a typical `launchSettings.json` file.
- ```
"profiles": {
```
- ```
  "IIS Express": {
```
- ```
 "commandName": "IISExpress",
```
- ```
    "launchBrowser": true,
```
- ```
 "environmentVariables": {
```
- ```
      "ASPNETCORE_ENVIRONMENT": "Development"
```
- ```
 }
```
- ```
  },
```

Multiple Environments and Development Mode

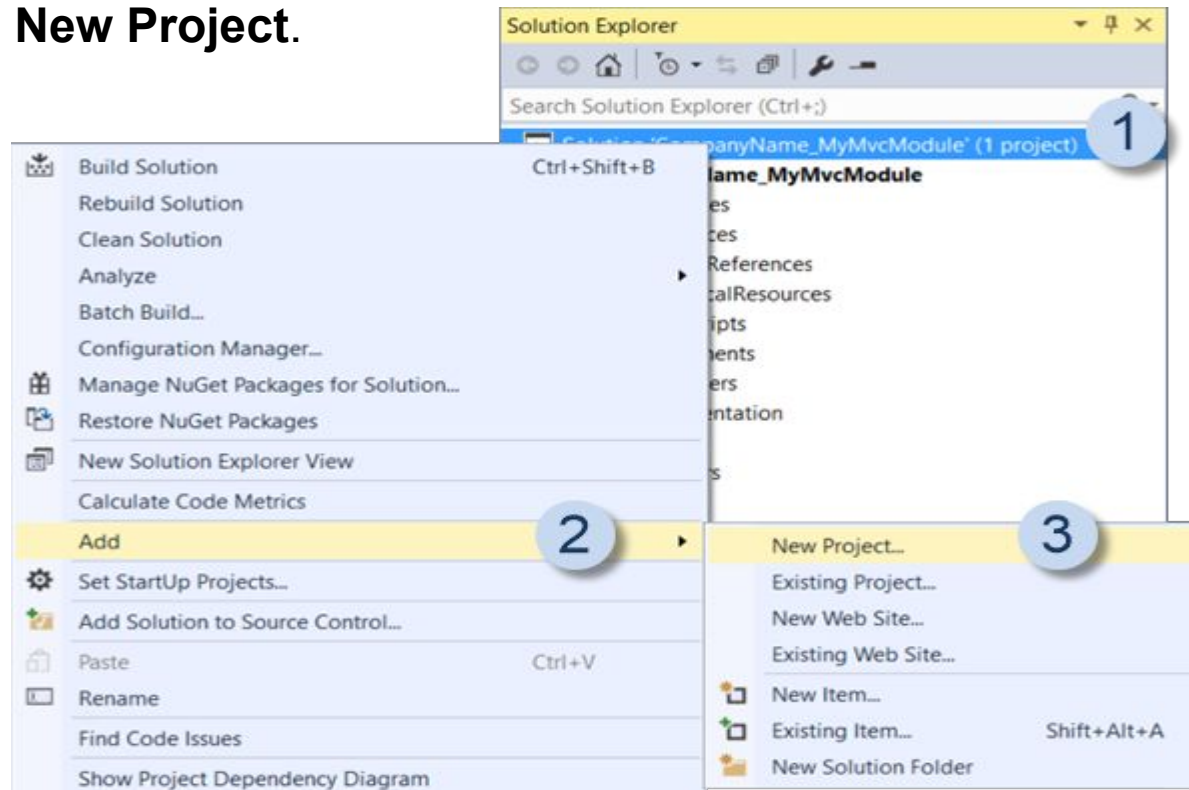
- "TutorialsPoint": {
- "commandName": "Project",
- "dotnetRunMessages": "true",
- "launchBrowser": true,
- "applicationUrl": "https://localhost:5001;http://localhost:5000",
- "environmentVariables": {
- "ASPNETCORE_ENVIRONMENT": "Development"
- }
- }
- }

Unit Test for an MVC module

- The primary goal of unit testing is to take the smallest piece of testable software in the application and determine whether it behaves exactly as you expect.
- Each unit is tested separately before integrating them into modules to test the interfaces between modules.

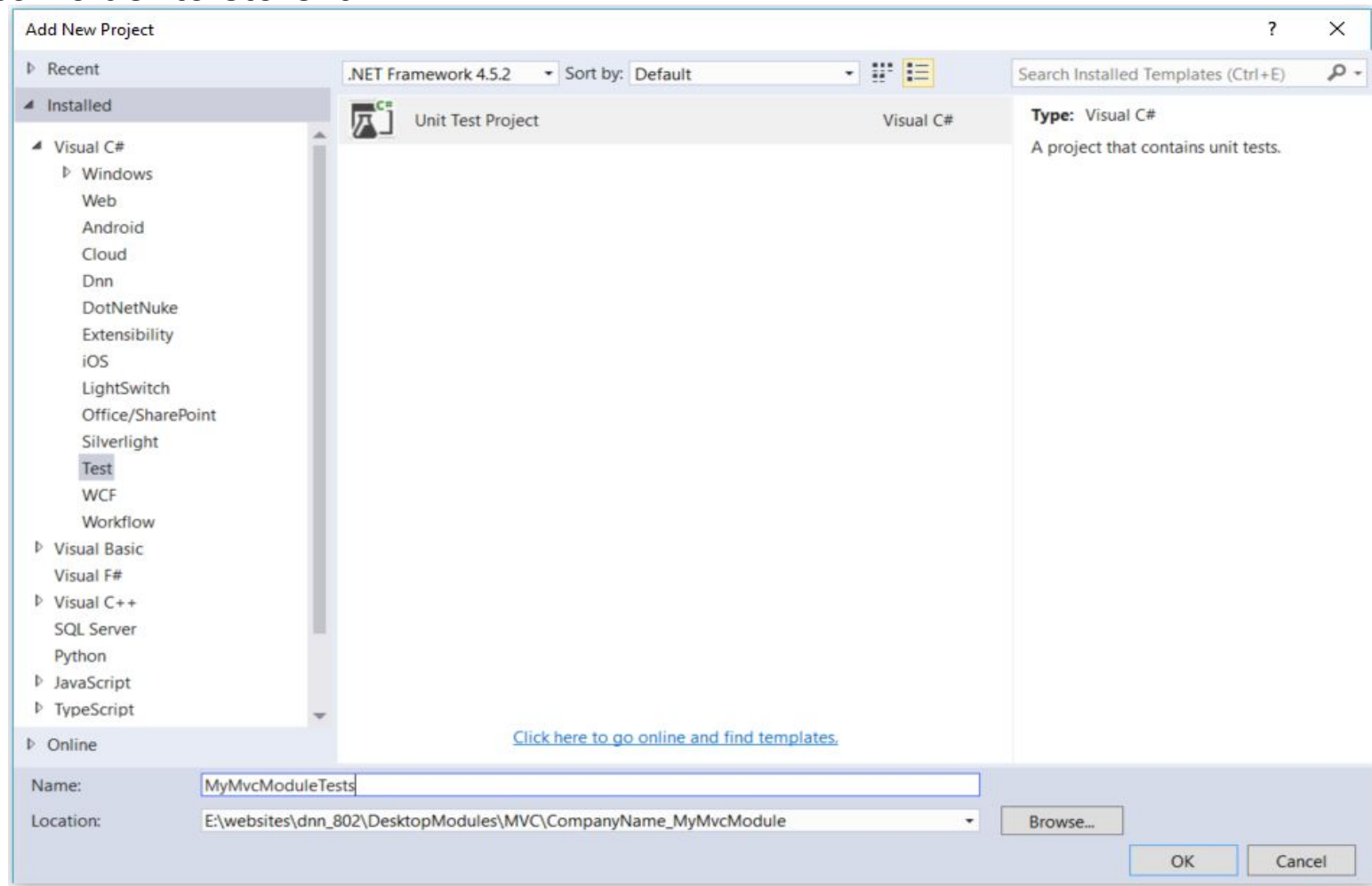
Unit Test for an MVC module

- Because MVC controllers are the central business logic of an MVC module, it is a best practice to create automated unit tests to ensure they behave as intended.
- **Steps**
- Add a new unit test project to the MVC module solution.
- In Visual Studio's **Solution Explorer**, right-click on your MVC module solution and select **Add > New Project**.



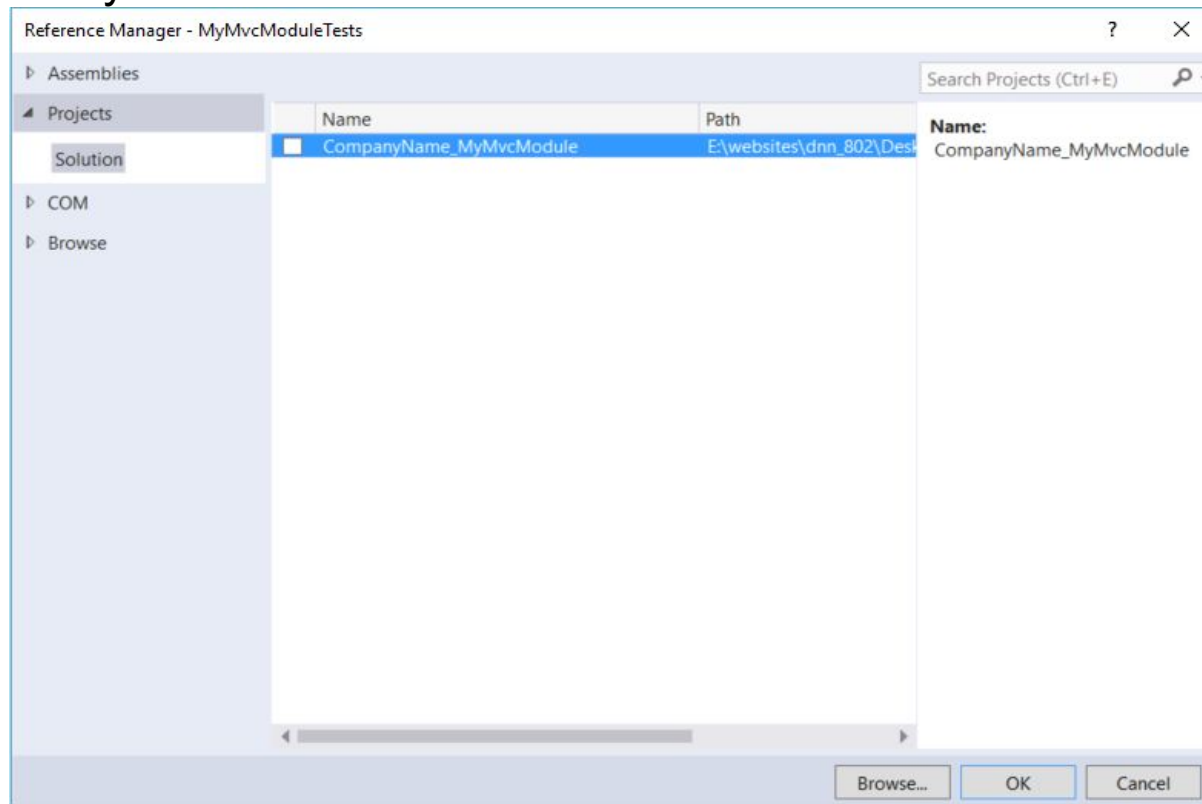
Unit Test for an MVC module

- In the **Add New Project** dialog, select **Unit Test Project**, enter a name, and select the local folder to store it in.



Unit Test for an MVC module

- Add the necessary MVC and DNN assembly references. For each assembly to be added to the new unit test project, right-click on the project's References node and add an assembly reference.



Unit Test for an MVC module

- Add references to the following assemblies, as well as others that your module specifically needs:
- DotNetNuke
- DotNetNuke.Web.Mvc
- System.Web.Mvc

Unit Test for an MVC module

Code to write in ItemControllerTests.cs

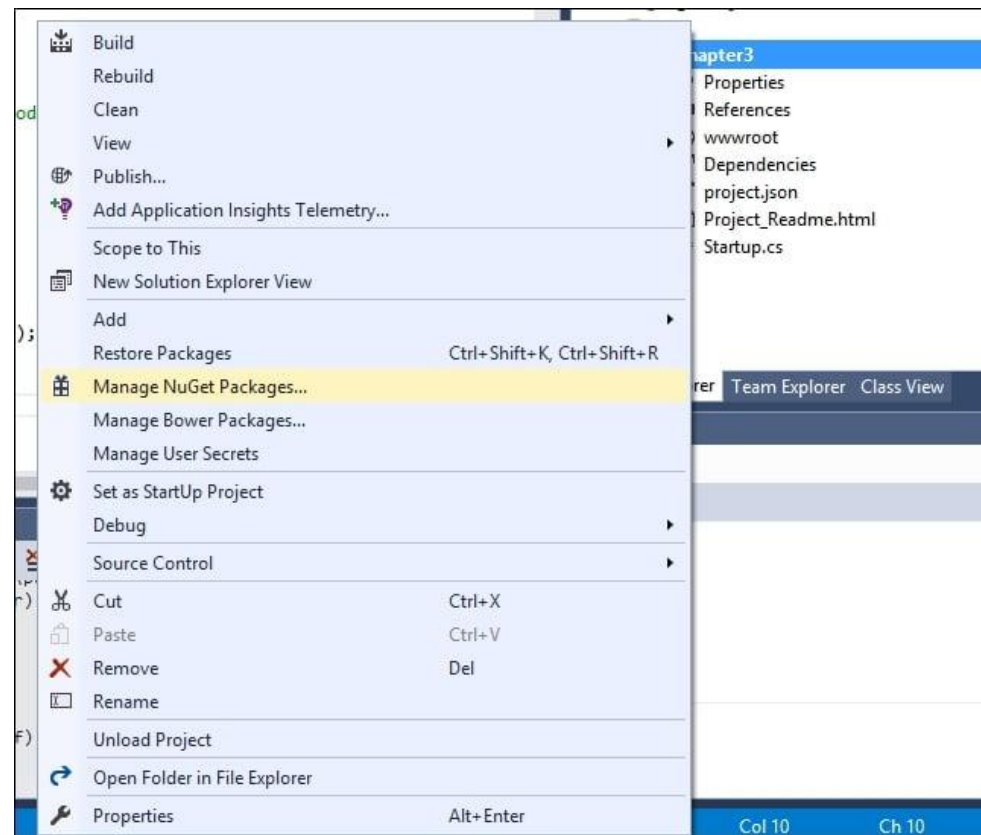
- [TestClass]
- public class ItemControllerTests
- {
- [TestMethod]
- public void Edit_CreateNewItem_ModuleIdAssignedinModel()
- {
- int moduleId = 2;
- var mockData = MockStores.MockItemManager();
- var modTwoItemCntrl = new ItemController(mockData.Object, moduleId); // Create a controller for the module with moduleId=2.
- // 2 - Act
- var actionResult = (ViewResult)modTwoItemCntrl.Edit(); // Call the edit view with no item Id
- // 3 - Assert
- var itemModel = (Item)actionResult.Model;
- Assert.IsTrue(itemModel != null && itemModel.ModuleId == moduleId);
- }

NuGet Package Manager

- NuGet can be used to find and install packages, that is, software pieces and assemblies and things that you want to use in your project.
- NuGet is not a tool that is specific to ASP.NET MVC projects. This is a tool that you can use inside of Visual Studio for console applications, WPF applications, Azure applications, any types of application.
- NuGet is a package manager, and is responsible for downloading, installing, updating, and configuring software in your system.
- NuGet is a package manager that delivers compiled source code (DLLs) and other files (scripts and images) related to code. A NuGet package takes the form of a zip file with the extension **.nupkg**. This makes adding, updating, and removing libraries easy in Visual Studio applications.

NuGet Package Manager

- Installing the ASP.NET Core NuGet package in your application
- Right click on the project, and select the **Manage NuGet Packages** option:



Navigation

- Navigation in ASP.NET Core MVC involves creating links and menus to help users navigate between different views or pages in your web application. Here are some common ways to handle navigation in ASP.NET Core MVC:
- **1. Using HTML `<a>` tags:**
- You can use standard HTML anchor tags to create links between different views. For example:
- `<a href="/Home/Index">Home</a>`
- `<a href="/Products/List">Products</a>`
- **2. Tag Helpers:**
- ASP.NET Core MVC provides tag helpers that simplify the process of creating links. The `asp-controller` and `asp-action` attributes help generate URLs based on the controller and action specified. For example:
- `<a asp-controller="Home" asp-action="Index">Home</a>`
- `<a asp-controller="Products" asp-action="List">Products</a>`

Navigation

- **3. Url.Action and Url.RouteUrl:**

- You can use Url.Action and Url.RouteUrl methods to generate URLs in the controller or views. For example, in a controller action method:
- ```
public IActionResult SomeAction()
```
- ```
{
```
- ```
 var url = Url.Action("Index", "Home");
```
- ```
}
```
- ```
Home
```

## 4. Navigation with Areas:

- If you are using areas in your application, you can specify the area in your link using the asp-area attribute:
- ```
<a asp-area="Admin" asp-controller="Dashboard" asp-action="Index">Admin Dashboard</a>
```

Navigation

- 5. RedirectToAction:
 - In your controller action methods, you can use the RedirectToAction method to redirect the user to another action. For example:
 - `public IActionResult SomeAction()`
 - `{`
 - `return RedirectToAction("Index", "Home");`
 - `}`
- 6. Navigation with TempData:
 - You can use TempData to pass data between actions, which can be useful for providing feedback or customizing the view based on the previous action.